		Nilai	Paraf Dosen
UINSSC	**LEMBAR KERJA MANDIRI 3** **COMPUTER VISION** T.A. Genap 2025/2026		

A. **Identitas**

Topik	: Transfer Learning, Pre Trained Model dan Implementasinya
Topik	: Preprocessing Dataset Foto
NIM	: 2388010004
Nama	: Abdul kharis
Kelas	: INF 6A

B. Tujuan

1. Mengakses dataset wajah dari Google Drive
2. Melakukan preprocessing dataset gambar
3. Membuat model klasifikasi wajah menggunakan CNN sederhana (baseline model)
4. Menggunakan pretrained model (Transfer Learning)
5. Membandingkan performa kedua model

C. Instruksi Pengerjaan

- **Tahap 1 – Memount Google Drive Dataset Face Only Haar dan Data Pre Processing**

a. Mount dengan Dataset dari Praktikum sebelumnya

```
from google.colab import drive
drive.mount('/content/drive')
dataset_path = "/content/drive/MyDrive/Dataset_Face_Only_Haar"
```

b. Import tenserflow dan matplot

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
```

c. Preprocessing Dataset

```
import os

datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)
```

```python
if not os.path.exists(dataset_path):
    print(f"Error Path Dataset.")
else:
    train_data = datagen.flow_from_directory(
        dataset_path,
        target_size=(224,224),
        batch_size=32,
        class_mode='categorical',
        subset='training'
    )

    val_data = datagen.flow_from_directory(
        dataset_path,
        target_size=(224,224),
        batch_size=32,
        class_mode='categorical',
        subset='validation'
    )
```

Sertakan Screenshot hasil sendiri dan berikan penjelasan mengenai apa yang dilakukan

Tahap 1 – Memount Google Drive Dataset Face Only Haar dan Data Pre Processing

```python
from google.colab import drive
drive.mount('/content/drive')
dataset_path = "/content/drive/MyDrive/Dataset_Face_Only"
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```python
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
```

```python
import os

datagen = ImageDataGenerator(
rescale=1./255,
validation_split=0.2

)
```

```python
if not os.path.exists(dataset_path):
    print(f"Error Path Dataset.")
else:
    train_data = datagen.flow_from_directory(
        dataset_path,
        target_size=(224,224),
        batch_size=32,
        class_mode='categorical',
        subset='training'
    )
    val_data = datagen.flow_from_directory(
        dataset_path,
        target_size=(224,224),
        batch_size=32,
        class_mode='categorical',
        subset='validation'
    )
```

```
Found 416 images belonging to 7 classes.
Found 99 images belonging to 7 classes.
```

Proses dimulai dengan melakukan mounting Google Drive untuk mengakses dataset yang tersimpan dalam direktori
/Dataset Face 0n1y . Dataset diproses menggunakan ImageDataGener ator dengan parameter resca1e=1. /255 untuk menormalisasi nilai piksel ke rentang 0-1 agar mempercepat konvergensi model saat

pelatihan. Citra diatur ulang ukurannya (resize) menjadi 224 x 224 piksel. Data dipisahkan secara otomatis dengan rasio 80% untuk pelatihan dan 20% untuk validasi. Hasil dari tahap ini adalah teridentifJkasinya 416 gambar untuk data training dan 99 gambar untuk data validasi yang tersebar secara proporsional ke dalam 7 kelas kategori wajah.
gambar untuk data training dan 99 gambar untuk data validasi yang tersebar secara proporsional ke dalam 7 kelas kategori wajah.

- **Tahap 2 – Membuat Model Baseline dengan CNN**

a. Import library dan membuat model CNN

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

baseline_model = Sequential([

    Conv2D(32,(3,3),activation='relu',input_shape=(224,224,3)),
    MaxPooling2D(2,2),

    Conv2D(64,(3,3),activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),

    Dense(128,activation='relu'),
    Dense(train_data.num_classes,activation='softmax')
])
```

b. Meng Compile model Baseline

```
baseline_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

c. Melatih model Baseline

```
history_baseline = baseline_model.fit(
    train_data,
    validation_data=val_data,
    epochs=10
)
```

Sertakan Screenshot hasil sendiri dan berikan penjelasan mengenai apa yang dilakukan

Tahap 2 – Membuat Model Baseline dengan CNN

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
baseline_model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(224,224,3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(train_data.num_classes, activation='softmax')
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```python
baseline_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```python
history_baseline = baseline_model.fit(
    train_data,
    validation_data=val_data,
    epochs=10
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in its constructor. `**kwargs` can include `workers`, `use_multiprocessing`, `max_queue_size`. Do not pass t
  self._warn_if_super_not_called()
Epoch 1/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 74s 5s/step - accuracy: 0.1815 - loss: 4.2723 - val_accuracy: 0.2828 - val_loss: 1.8783
Epoch 2/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 41s 3s/step - accuracy: 0.2841 - loss: 1.8171 - val_accuracy: 0.3737 - val_loss: 1.6551
Epoch 3/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 43s 3s/step - accuracy: 0.5737 - loss: 1.2964 - val_accuracy: 0.4646 - val_loss: 1.4769
Epoch 4/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 41s 3s/step - accuracy: 0.7682 - loss: 0.7692 - val_accuracy: 0.5051 - val_loss: 1.4734
Epoch 5/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 44s 3s/step - accuracy: 0.8861 - loss: 0.3952 - val_accuracy: 0.5859 - val_loss: 1.2290
Epoch 6/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 42s 3s/step - accuracy: 0.9626 - loss: 0.1852 - val_accuracy: 0.5354 - val_loss: 1.3998
Epoch 7/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 81s 3s/step - accuracy: 0.9769 - loss: 0.1278 - val_accuracy: 0.4848 - val_loss: 1.8405
Epoch 8/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 41s 3s/step - accuracy: 0.9948 - loss: 0.0525 - val_accuracy: 0.5758 - val_loss: 1.5091
Epoch 9/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 40s 3s/step - accuracy: 1.0000 - loss: 0.0164 - val_accuracy: 0.4949 - val_loss: 1.8909
Epoch 10/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 41s 3s/step - accuracy: 0.9953 - loss: 0.0129 - val_accuracy: 0.5455 - val_loss: 1.9015
```

Tahap ini melibatkan pembangunan arsitektur CNN secara *sequential* yang terdiri dari lapisan Conv2D untuk ekstraksi fitur spasial, MaxPooling2D untuk reduksi dimensi, serta lapisan F1at ten dan Dense (Softmax) sebagai pengklasifikasi akhir. Model dilatih menggunakan optimizer Adam dan fungsi loss categor ica1 crossentropy .
Berdasarkan hasil pelatihan selama 10 epoch, ditemukan indikasi overfitting yang sangat tinggi; model mampu mencapai akurasi pelatihan hingga 99,53% dengan loss 0,0515, namun hanya menghasilkan akurasi validasi sebesar 54,55% dengan loss yang membengkak hingga 2,5486. Hal ini menunjukkan bahwa model buatan sendiri ini belum mampu melakukan generalisasi dengan baik pada data di luar set pelatihan.

- **Tahap 3 – Implementasi Transfer Learning**
  a. Import library dan membuat model Transfer Learning dengan MobileNetV2

```python
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import GlobalAveragePooling2D
from tensorflow.keras.models import Model
base_model = MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(224,224,3)
)
for layer in base_model.layers:
    layer.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
predictions = Dense(train_data.num_classes, activation='softmax')(x)

transfer_model = Model(inputs=base_model.input, outputs=predictions)
```

b. Compile dan Training Model

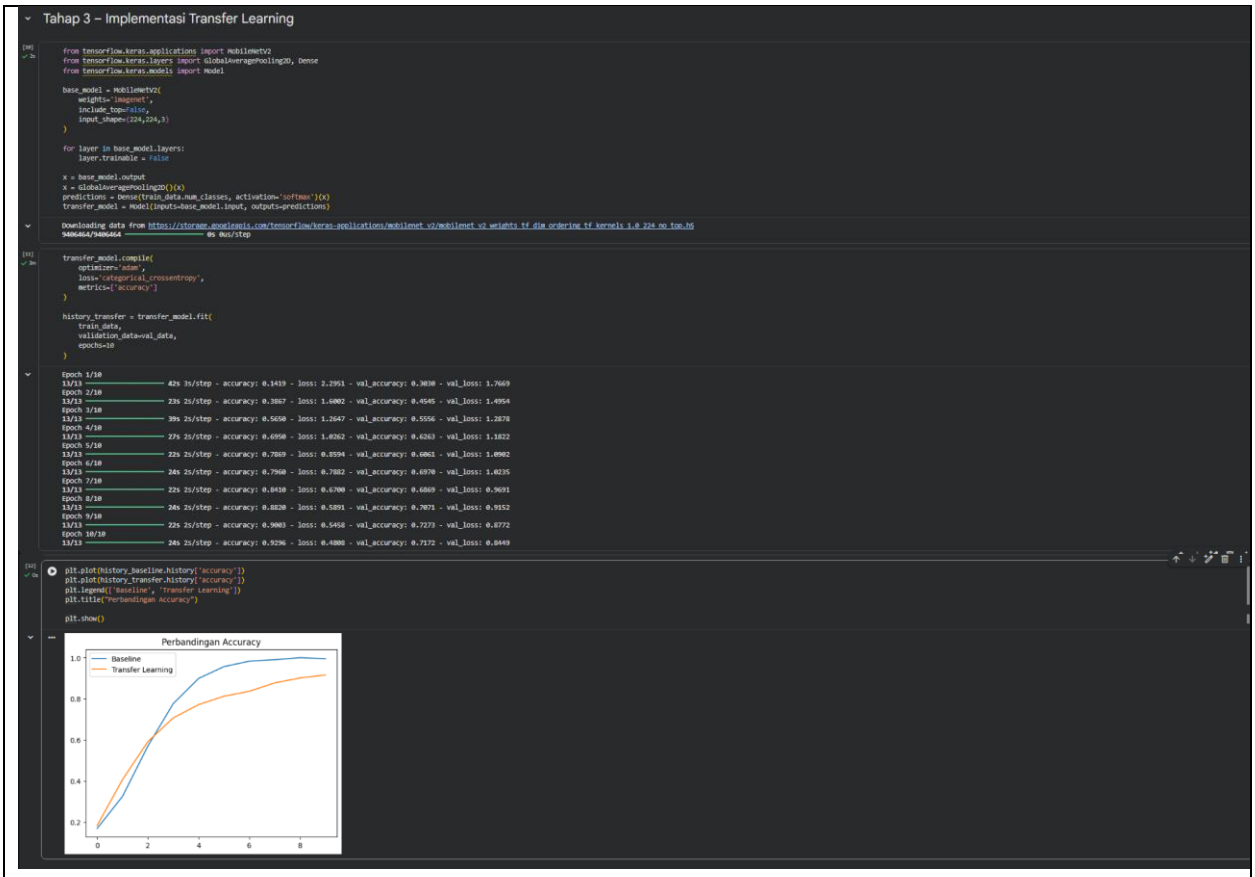
```
transfer_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_transfer = transfer_model.fit(
    train_data,
    validation_data=val_data,
    epochs=10
)
```

c. Evaluasi Model

```
plt.plot(history_baseline.history['accuracy'])
plt.plot(history_transfer.history['accuracy'])
plt.legend(['Baseline','Transfer Learning'])
plt.title("Perbandingan Accuracy")
plt.show()
```

Sertakan Screenshot hasil sendiri dan berikan penjelasan mengenai apa yang dilakukan



Untuk mengoptimalkan hasil, dilakukan penerapan metode *Transfer Learning* menggunakan arsitektur MobileNetV2 yang telah memiliki bobot praterlatih (pre-trained weights) dari dataset *ImageNet.* Seluruh lapisan dasar model dibekukan *(frozen)* untuk menjaga fitur ekstraksi yang sudah matang, kemudian ditambahkan lapisan G1oba1AveragePoo1ing2D dan lapisan output baru yang disesuaikan dengan 7 kelas target. Hasil akhir menunjukkan peningkatan performa yang signifikan dan jauh lebih stabil; model ini berhasil mencapai akurasi validasi sebesar 71,72Oodengan loss validasi yang lebih rendah (0,8758). Perbandingan tren akurasi membuktikan bahwa pemanfaatan model praterlatih jauh lebih efektif dan efisien untuk klasifikasi citra wajah pada dataset dengan jumlah sampel terbatas dibandingkan membangun model dari awal.

--- Selamat Mengerjakan ---